```
import streamlit as st
import yfinance as yf
import pandas as pd
from datetime import date

st.set_page_config(page_title="MA Backtester", layout="wide")
st.title("Moving Average Backtester")
st.caption("A learning tool, not financial advice. Past results don't predict fut

# ---- Inputs ----
ticker = st.text_input("Ticker", "SPY").strip().upper()

col_a, col_b = st.columns(2)
start = col_a.date_input("Start date", date(2015, 1, 1))
end = col_b.date_input("End date", date(2024, 12, 31))

col_c, col_d = st.columns(2)
fast = col_c.number_input("Fast window (days)", min_value=5, max_value=200, value
slow = col_d.number_input("Slow window (days)", min_value=20, max_value=400, valu

cost_pct = st.slider("Cost per trade (% of position value)", 0.0, 1.0, 0.10, 0.01


# ---- Data ----
@st.cache_data(show_spinner="Downloading prices...")
def load_prices(ticker, start, end):
    df = yf.download(ticker, start=start, end=end, auto_adjust=True, progress=Fal
    # Newer yfinance versions return multi-level column names; flatten them
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)
    return df[["Close"]].dropna()


def max_drawdown(growth):
    """Largest peak-to-trough drop, as a negative percentage."""
    return (growth / growth.cummax() - 1).min()


# ---- Checks ----
if fast >= slow:
    st.warning("The fast window should be shorter than the slow window.")
elif start >= end:
    st.warning("The end date must be after the start date.")
else:
```

```
prices = load_prices(ticker, start, end)

if prices.empty:
    st.error("No price data found. Check the ticker symbol and dates.")
else:
    data = prices.copy()
    close = data["Close"]

    # Signals: hold (1) when the fast average is above the slow average
    data["SMA_fast"] = close.rolling(fast).mean()
    data["SMA_slow"] = close.rolling(slow).mean()
    data["Position"] = (data["SMA_fast"] > data["SMA_slow"]).astype(int).shif

    # Returns
    market_ret = close.pct_change().fillna(0)
    trades = data["Position"].diff().abs().fillna(0)  # 1 whenever we buy or
    costs = trades * (cost_pct / 100)
    strategy_ret = data["Position"] * market_ret - costs

    data["Buy & hold"] = (1 + market_ret).cumprod()
    data["Strategy"] = (1 + strategy_ret).cumprod()

    # ---- Results ----
    bh_total = data["Buy & hold"].iloc[-1] - 1
    strat_total = data["Strategy"].iloc[-1] - 1
    bh_dd = max_drawdown(data["Buy & hold"])
    strat_dd = max_drawdown(data["Strategy"])
    num_trades = int(trades.sum())

    c1, c2, c3 = st.columns(3)
    c1.metric("Buy & hold return", f"{bh_total:.1%}")
    c2.metric("Strategy return", f"{strat_total:.1%}",
              delta=f"{strat_total - bh_total:+.1%} vs buy & hold")
    c3.metric("Trades placed", num_trades)

    c4, c5 = st.columns(2)
    c4.metric("Buy & hold max drawdown", f"{bh_dd:.1%}")
    c5.metric("Strategy max drawdown", f"{strat_dd:.1%}")

    st.subheader("Growth of 1 unit of money")
    st.line_chart(data[["Buy & hold", "Strategy"]])

    st.subheader("Drawdown over time")
    dd = pd.DataFrame({
        "Buy & hold": data["Buy & hold"] / data["Buy & hold"].cummax() - 1,
        "Strategy": data["Strategy"] / data["Strategy"].cummax() - 1,
    })
```

```
st.line_chart(dd)

if strat_total > bh_total:
    st.info("The strategy beat buy & hold after costs in this test. "
            "Now check other tickers and date ranges before trusting it."
else:
    st.info("The strategy did not beat buy & hold after costs in this tes
            "That's a common result.")
```